

3D Computer Vision

Camera Model & Calibration

Prof. Didier Stricker

Kaiserslautern University

<http://ags.cs.uni-kl.de/>

DFKI – Deutsches Forschungszentrum für Künstliche Intelligenz

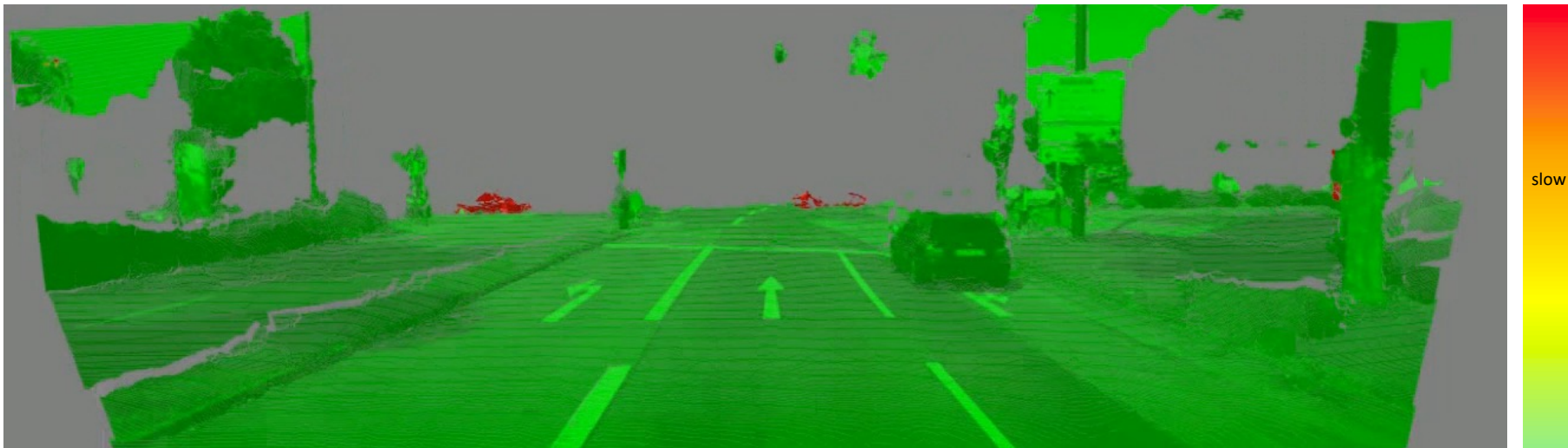
<http://av.dfki.de>

Appetizer: Scene Flow

View of one camera of the stereo camera



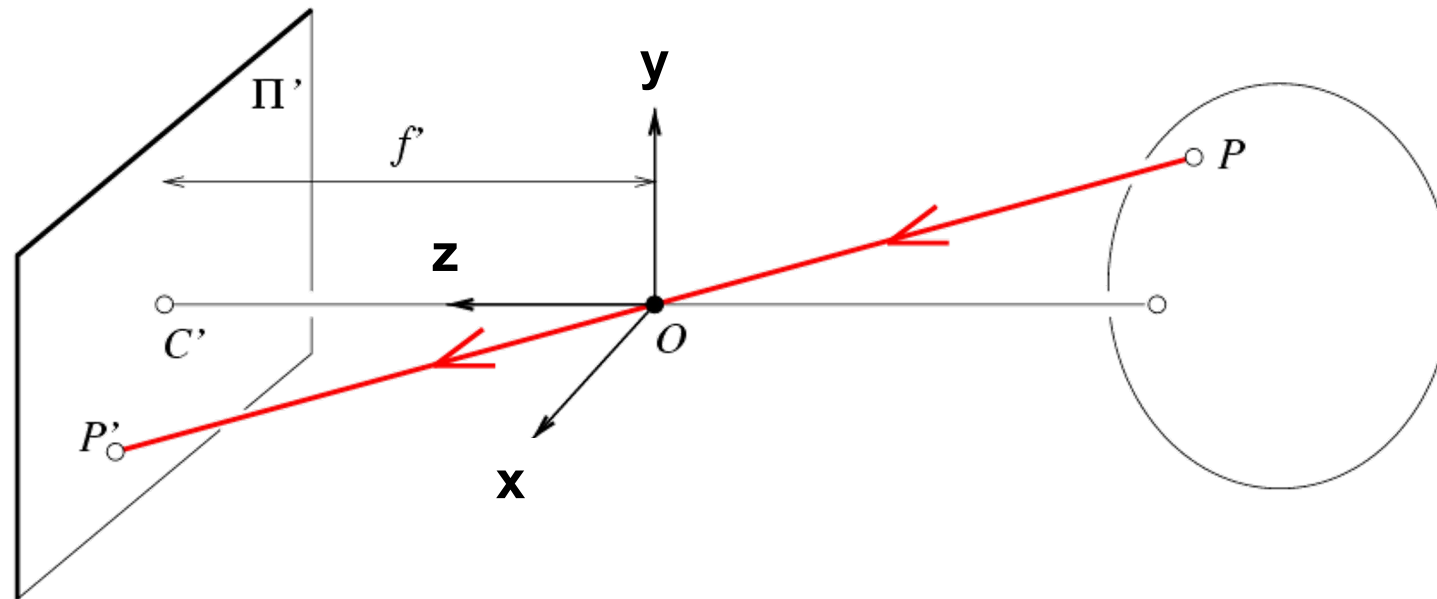
Exemplary stereo camera



Scene reconstructed as **dynamic** 3D point cloud

- Homogenous coordinates and central linear projection
- Camera parameters
 - Intrinsic parameters
 - Extrinsic parameters
- Calibration procedure
 - Computing camera matrix P
 - Extracting extrinsic parameters R, t
 - Extracting calibration matrix K
 - Advanced calibration
- Representation of 3D rotation

Recall: Modeling projection



•Projection equations

- Compute intersection with Π' of ray from $\mathbf{P} = (x,y,z)$ to $\mathbf{O}$
- Derived using similar triangles
- We get the projection by throwing out the last coordinate:

$$(x, y, z) \rightarrow \left(f' \frac{x}{z}, f' \frac{y}{z}\right)$$

Homogeneous coordinates

$$(x, y, z) \rightarrow (f' \frac{x}{z}, f' \frac{y}{z})$$

- Is this a linear transformation?
 - no—division by z is nonlinear

Trick: add one more coordinate:

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

homogeneous image
coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

homogeneous scene
coordinates

Converting *from* homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w) \qquad \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

Perspective Projection Matrix

- Projection is a matrix multiplication using homogeneous coordinates:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f' & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z/f' \end{bmatrix}$$

$$\Rightarrow \left(f' \frac{x}{z}, f' \frac{y}{z} \right)$$

divide by the third
coordinate

Perspective Projection Matrix

- Projection is a matrix multiplication using homogeneous coordinates:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f' & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z/f' \end{bmatrix} \Rightarrow \left(f' \frac{x}{z}, f' \frac{y}{z} \right)$$

divide by the third coordinate

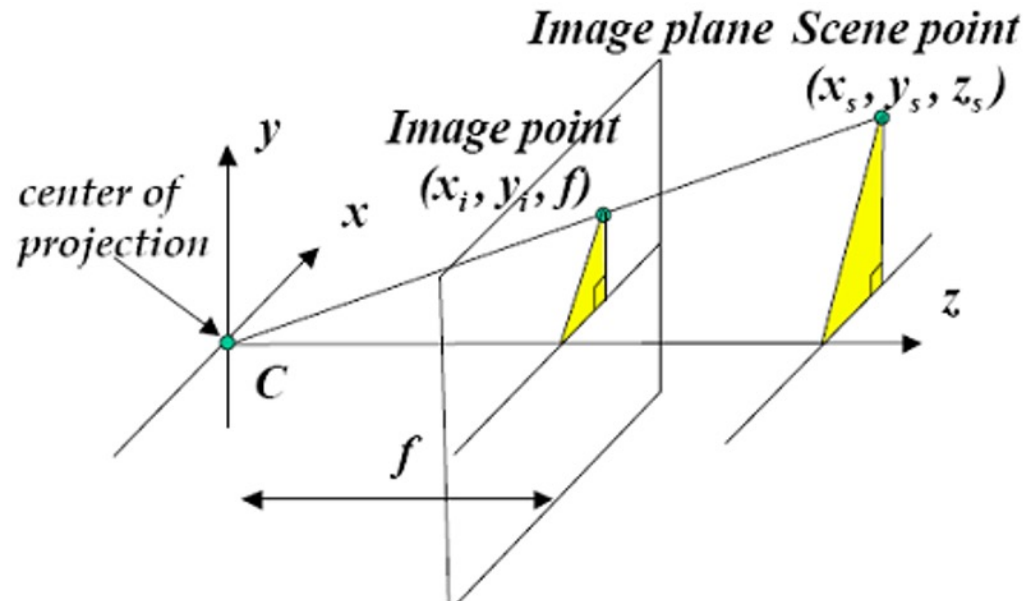
In practice: lots of coordinate transformations...

$$\begin{pmatrix} \text{2D point} \\ (3 \times 1) \end{pmatrix} = \begin{pmatrix} \text{Camera to pixel coord. trans. matrix} \\ (3 \times 3) \end{pmatrix} \begin{pmatrix} \text{Perspective projection matrix} \\ (3 \times 4) \end{pmatrix} \begin{pmatrix} \text{World to camera coord. trans. matrix} \\ (4 \times 4) \end{pmatrix} \begin{pmatrix} \text{3D point} \\ (4 \times 1) \end{pmatrix}$$

Perspective Projection

For simplicity, we use the virtual **image plane in front of the camera**.

- Same results in the image plane (same image) but
- All variables/values are positive, ie. in front of the camera center and the horizontal flip of the image (what is simpler)



$$x_i = f \frac{x_s}{z_s}$$

$$y_i = f \frac{y_s}{z_s}$$

Using homogeneous coordinates, the perspective projection becomes linear!

$$\begin{pmatrix} u \\ v \\ w \end{pmatrix} = \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} \quad \rightarrow \quad \begin{aligned} x_i &= \frac{u}{w} \\ y_i &= \frac{v}{w} \end{aligned}$$

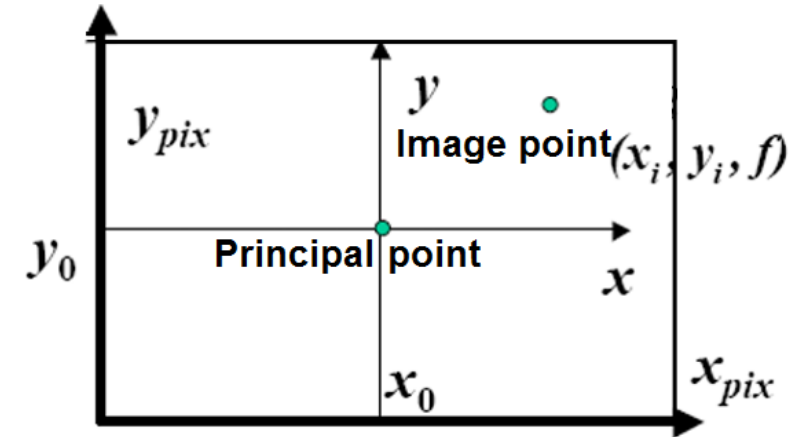
- Homogenous coordinates and central linear projection
- Camera parameters
 - Intrinsic parameters
 - Extrinsic parameters
- Calibration procedure
 - Computing camera matrix P
 - Extracting extrinsic parameters R, t
 - Extracting calibration matrix K
 - Advanced calibration
- Representation of 3D rotation

Intrinsic Parameters – Pixel Coordinates

- **Transformation uses:**

- Principal point (x_0, y_0)
- Scaling factors k_x and k_y

$$\begin{aligned}x_i &= f \frac{x_s}{z_s} & x_{pix} &= k_x x_i + x_0 = \frac{f k_x x_s + z_s x_0}{z_s} \\y_i &= f \frac{y_s}{z_s} & y_{pix} &= k_y y_i + y_0 = \frac{f k_y y_s + z_s y_0}{z_s}\end{aligned}$$



And in homogeneous coordinates with $\alpha_x = f k_x$ and $\alpha_y = f k_y$:

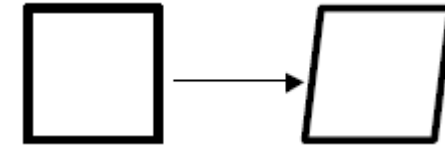
$$\begin{pmatrix} u' \\ v' \\ w' \end{pmatrix} = \begin{pmatrix} \alpha_x & 0 & x_0 & 0 \\ 0 & \alpha_y & y_0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} \rightarrow \begin{aligned}x_{pix} &= \frac{u'}{w'} \\ y_{pix} &= \frac{v'}{w'}\end{aligned}$$

Intrinsic Parameters - Calibration Matrix K

$$\begin{pmatrix} u' \\ v' \\ w' \end{pmatrix} = \begin{pmatrix} \alpha_x & s & x_0 & 0 \\ 0 & \alpha_y & y_0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} \rightarrow \begin{aligned} x_{pix} &= \frac{u'}{w'} \\ y_{pix} &= \frac{v'}{w'} \end{aligned}$$

- **Intrinsic parameters:**

- $\alpha_x = fk_x$ and $\alpha_y = fk_y$: “Focal lengths” in pixels
- x_0 and y_0 : Coordinates of principal point in pixels
- Added parameter s is skew parameter

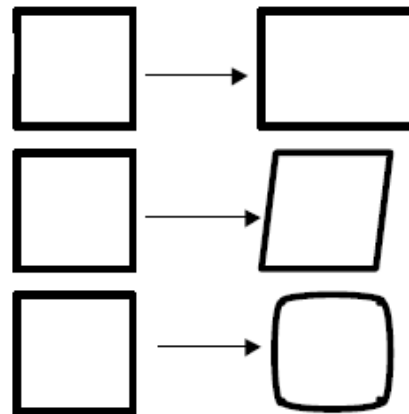


$$\begin{pmatrix} \alpha_x & s & x_0 & 0 \\ 0 & \alpha_y & y_0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} \alpha_x & s & x_0 \\ 0 & \alpha_y & y_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} = K[I_3|0_3]$$

- **K is called calibration matrix:**

- Five degrees of freedom
- K is a 3x3 upper triangular matrix

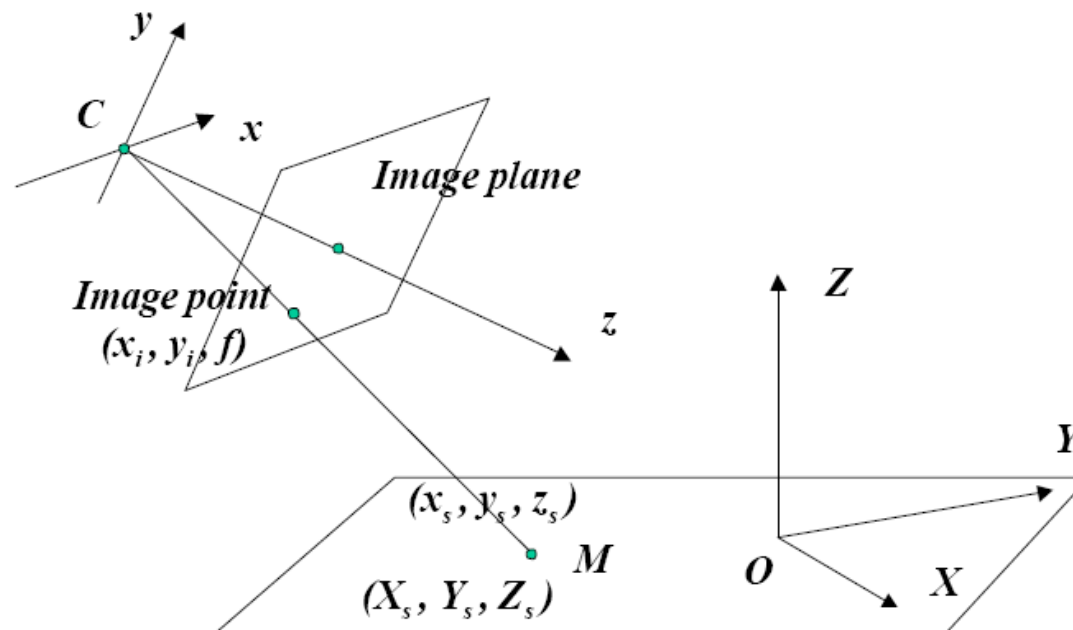
- **Finding camera's internal / intrinsic parameters that effect the imaging process**
 - Position of the principal point on the image (x_0, y_0) , in the optimal case this is the image center (width/2, height/2).
 - Focal length f .
 - Scaling factors for row and column pixels k_x and k_y .
 - Skew parameter s .
 - Lens distortion.



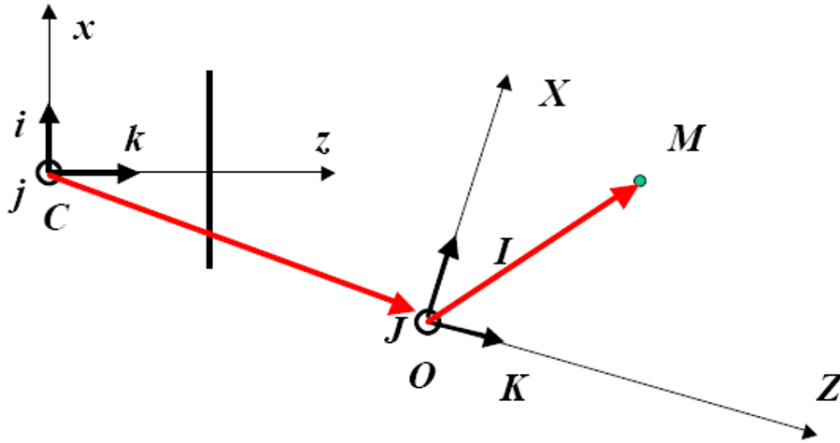
Extrinsic Parameters

– World Coordinates and Camera Coordinates

- **In general world coordinates and camera coordinates do not coincide.**
→ We need to find a coordinate transformation.



Extrinsic Parameters – World Frame to Camera Frame



- $M = (X_s, Y_s, Z_s)$: point in world coordinates
- $M = (x_s, y_s, z_s)$: point in camera coordinates
- $t = CO$: translation vector between origins

$$CM = CO + OM$$

$$x_s \mathbf{i} + y_s \mathbf{j} + z_s \mathbf{k} = t_x \mathbf{i} + t_y \mathbf{j} + t_z \mathbf{k} + X_s \mathbf{I} + Y_s \mathbf{J} + Z_s \mathbf{K}$$

$$\Rightarrow x_s = t_x + X_s \mathbf{I} \cdot \mathbf{i} + Y_s \mathbf{J} \cdot \mathbf{i} + Z_s \mathbf{K} \cdot \mathbf{i}$$

$$\Rightarrow \begin{pmatrix} x_s \\ y_s \\ z_s \end{pmatrix} = \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix} + \begin{pmatrix} \mathbf{I} \cdot \mathbf{i} & \mathbf{J} \cdot \mathbf{i} & \mathbf{K} \cdot \mathbf{i} \\ \mathbf{I} \cdot \mathbf{j} & \mathbf{J} \cdot \mathbf{j} & \mathbf{K} \cdot \mathbf{j} \\ \mathbf{I} \cdot \mathbf{k} & \mathbf{J} \cdot \mathbf{k} & \mathbf{K} \cdot \mathbf{k} \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \end{pmatrix}$$

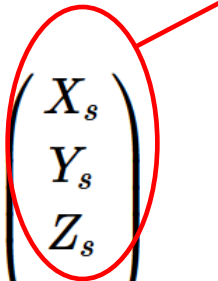
Rotation matrix

Extrinsic Parameters – Coordinate Transformation

$$\begin{pmatrix} x_s \\ y_s \\ z_s \end{pmatrix} = \begin{pmatrix} t_x \\ t_y \\ t_s \end{pmatrix} + \begin{pmatrix} \mathbf{I}.i & \mathbf{J}.i & \mathbf{K}.i \\ \mathbf{I}.j & \mathbf{J}.j & \mathbf{K}.j \\ \mathbf{I}.k & \mathbf{J}.k & \mathbf{K}.k \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \end{pmatrix}$$

Using homogeneous coordinates:

$$\begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{I}.i & \mathbf{J}.i & \mathbf{K}.i & t_x \\ \mathbf{I}.j & \mathbf{J}.j & \mathbf{K}.j & t_y \\ \mathbf{I}.k & \mathbf{J}.k & \mathbf{K}.k & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \\ 1 \end{pmatrix} = \begin{pmatrix} R & t \\ 0_3^\top & 1 \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \\ 1 \end{pmatrix}$$

$\tilde{C}$ 

And with $\tilde{C} = -R^T t$ (C is the camera center):

$$\begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} = \begin{pmatrix} R & -R\tilde{C} \\ 0_3^\top & 1 \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \\ 1 \end{pmatrix}$$

Camera Matrix P – Putting everything together

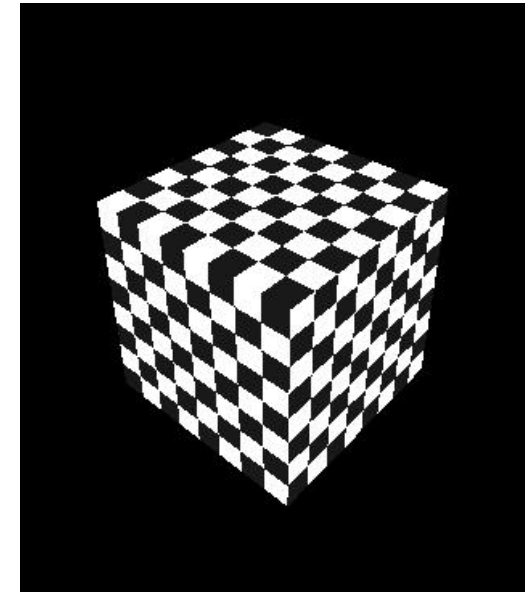
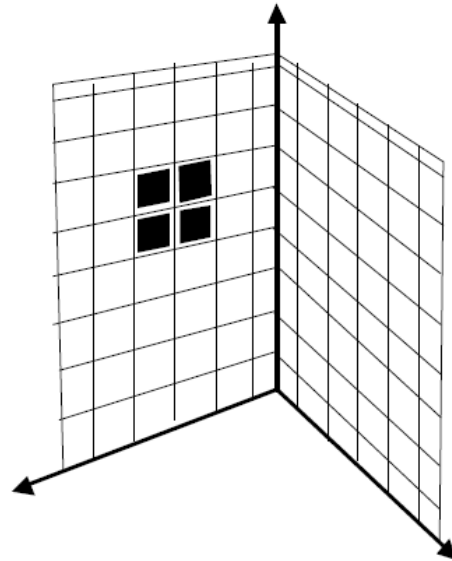
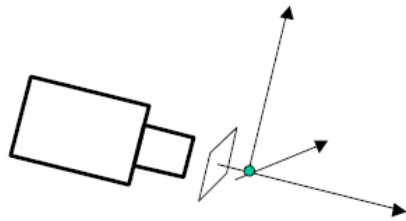
$$\begin{aligned} \begin{pmatrix} u' \\ v' \\ w' \end{pmatrix} &= K(I_3 | 0_3) \begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} \\ \begin{pmatrix} x_s \\ y_s \\ z_s \\ 1 \end{pmatrix} &= \begin{pmatrix} R & -R\tilde{C} \\ 0_3^\top & 1 \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \\ 1 \end{pmatrix} \end{aligned} \quad \Rightarrow \quad \begin{pmatrix} u' \\ v' \\ w' \end{pmatrix} = K(I_3 | 0_3) \begin{pmatrix} R & -R\tilde{C} \\ 0_3^\top & 1 \end{pmatrix} \begin{pmatrix} X_s \\ Y_s \\ Z_s \\ 1 \end{pmatrix} \\ &= P \begin{pmatrix} X_s \\ Y_s \\ Z_s \\ 1 \end{pmatrix}$$

$$x = PX$$

- Homogenous coordinates and central linear projection
- Camera parameters
 - Intrinsic parameters
 - Extrinsic parameters
- Calibration procedure
 - Computing camera matrix P
 - Extracting extrinsic parameters R, t
 - Extracting calibration matrix K
 - Advanced calibration
- Representation of 3D rotation

Calibration – Procedure

- **Calibration means finding the function (defined by the camera) that maps 3D points to 2D image plane.**
 - First thing required is to obtain pairs of corresponding 3D and 2D points:
 $(X_1, Y_1, Z_1), (x_1, y_1), (X_2, Y_2, Z_2), (x_2, y_2), \dots$

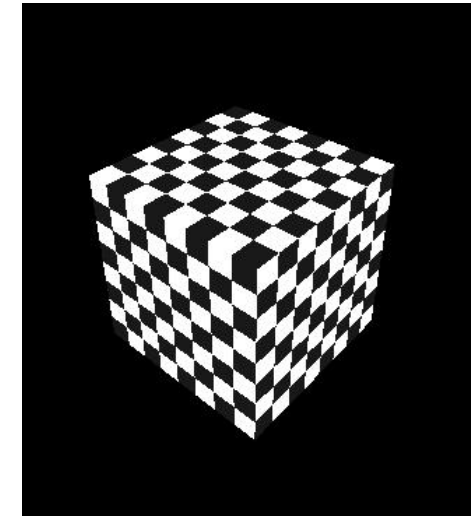


Calibration – Calibration Target

- **Calibration target: Two perpendicular planes with chessboard pattern.**
 1. We **know the 3D positions** of the corners with respect to a coordinates system defined on the target.
 2. Place a camera in front of the target and we can locate the corresponding corners on the image. This gives us the **correspondences**.
 3. Recover the **equation** that describes imaging projection and camera's internal parameters. At the same time, also recover the relative orientation between the camera and the target (pose).

Calibration – Finding Correspondences

- **Multiple ways to find correspondences:**
 - Corner detector.
 - Canny Edge detector plus fitting lines to the detected edges. Find the intersections of the lines
 - Manual input.
- **Matching 3D and 2D points (we know the number of corners) by counting gives corresponding point pairs.**
 - $(X_1, Y_1, Z_1), (x_1, y_1), (X_2, Y_2, Z_2), (x_2, y_2), \dots$



Calibration – Computing the Camera Matrix P

- **Set up equations for all corresponding point pairs:**

- $\mathbf{x}_i = P\mathbf{X}_i, \quad \mathbf{x}_i = (x_i, y_i), \quad \mathbf{X}_i = (X_i, Y_i, Z_i), \quad i = 1, \dots$

- Involving homogeneous coordinates $\mathbf{x}_i$ and $P\mathbf{X}_i$ just have to be proportional:

$$\mathbf{x}_i \times P\mathbf{X}_i = 0, \quad i = 1, \dots \quad (\times \text{ being the cross product})$$

Calibration – Computing the Camera Matrix P

- Let $p_1^\top, p_2^\top, p_3^\top$ be the row vectors of matrix P .

$$P\mathbf{X}_i = \begin{pmatrix} p_1^\top \mathbf{X}_i \\ p_2^\top \mathbf{X}_i \\ p_3^\top \mathbf{X}_i \end{pmatrix} \quad \mathbf{x}_i \times P\mathbf{X}_i = \begin{pmatrix} v'_i p_3^\top \mathbf{X}_i - w'_i p_2^\top \mathbf{X}_i \\ w'_i p_1^\top \mathbf{X}_i - u'_i p_3^\top \mathbf{X}_i \\ u'_i p_2^\top \mathbf{X}_i - v'_i p_1^\top \mathbf{X}_i \end{pmatrix}$$

- We get the linear system of equations:

$$\Rightarrow \begin{pmatrix} 0_4^\top & -w'_i \mathbf{X}_i^\top & v'_i \mathbf{X}_i^\top \\ w'_i \mathbf{X}_i^\top & 0_4^\top & -u'_i \mathbf{X}_i^\top \\ -v'_i \mathbf{X}_i^\top & u'_i \mathbf{X}_i^\top & 0_4^\top \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix} = 0_{12}$$

Calibration – Computing the Camera Matrix P

- The third row can be obtained from the sum of u'_i times the first row and $-v'_i$ times the second row.

$$\begin{pmatrix} 0_4^\top & -w'_i \mathbf{X}_i^\top & v'_i \mathbf{X}_i^\top \\ w'_i \mathbf{X}_i^\top & 0_4^\top & -u'_i \mathbf{X}_i^\top \\ -v'_i \mathbf{X}_i^\top & u'_i \mathbf{X}_i^\top & 0_4^\top \end{pmatrix} \begin{pmatrix} p_1 \\ p_2 \\ p_3 \end{pmatrix} = 0_{12}$$

Rank 2

- So we get 2 independent equations in 11 unknowns (ignoring scale).
- With at 6 point correspondences we have enough equations to compute matrix P by solving:

$$A\mathbf{p} = 0$$

Calibration – Computing the Camera Matrix P

- Linear system.
- If possible, have at least 5 times as many equations as unknowns (28 points).
- Minimize $\|A\mathbf{p}\|_2^2$ with the constraint $\|\mathbf{p}\|_2 = 1$ to avoid the trivial solution.
 - The minimizer $\mathbf{p}$ of this kind of minimization problem is given by the unit singular vector of A corresponding to the smallest singular value ($\neq 0$).

Usually, it is the last column of V , if the SVD is given by:

$$A = USV^T$$

Calibration – Extracting Extrinsics

- **Having computed P we can extract the extrinsic parameters from**

$$P = KR(I_3 | -\tilde{C})$$

- The left 3x3 sub-matrix is a product of an upper triangular matrix K and an orthogonal matrix R .

- **First, we want to find homogeneous coordinates of C in the scene:**

- C is the null-vector of matrix P :

$$\begin{aligned} PC &= 0_3 \\ \Leftrightarrow P &= KR(I_3 | -\tilde{C}) \end{aligned} \quad \Leftrightarrow \quad \begin{pmatrix} 1 & 0 & 0 & -X_C \\ 0 & 1 & 0 & -Y_C \\ 0 & 0 & 1 & -Z_C \end{pmatrix} \begin{pmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{pmatrix} = 0_3$$

- Find null-vector C of P using SVD again.

- **Next we want to find the camera orientation and the internal parameters of the camera:**

- K and an upper triangular matrix
- R is an orthogonal matrix.

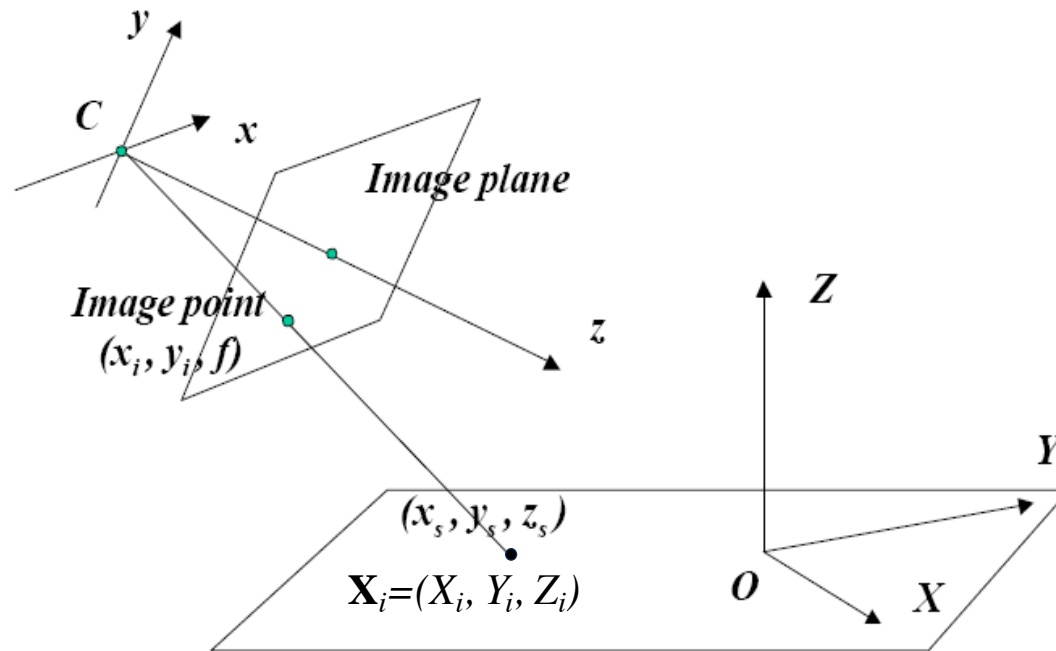
→ From the first 3x3 sub-matrix of $P = KR$ we can extract K and R by **QR-Decomposition**.

- A QR-decomposition (also called QR-factorization) of a matrix is a decomposition into an orthogonal (Q) and a right triangular matrix (R).
- In our case we use RQ decomposition, which is similar to QR (apply QR decomposition on inverted input and invert output).

$$\boxed{KR} = ((\boxed{KR})^{-1})^{-1} = (\boxed{Q} \begin{array}{c} \diagup \\ R \end{array})^{-1} = \begin{array}{c} \diagdown \\ R^{-1} \end{array} \boxed{Q}^{-1}$$

Calibration – Degenerate Case

Calibration points $\mathbf{X}_i$, $i = 1, \dots$ can not lie on the same plane!



In this case there would exist a mapping with only 8 degrees of freedom (called **homography** → next lecture).

Fitting a P -Matrix with 11 DOF is not uniquely defined!

Calibration – Back-Projection Error

- **How should we estimate the accuracy of a calibration algorithm?**
 - Project known 3D points on the image.
 - Compare their projections with the corresponding pixel coordinates of the points.
 - Repeat for many points and estimate **back-projection / re-projection** error!

- **How can we further improve the calibration?**
 - Use received linear calibration as initialization for nonlinear methods for refinement.
 - Minimize the nonlinear error function (usually back-projection error):

$$\sum_{P, \mathbf{x}_i, \mathbf{X}_i} d(\mathbf{x}_i, P\mathbf{X}_i)^2$$

- Most popular non-linear minimization algorithm is the **Levenberg-Marquart (LM)** minimization
- LM is more robust to local minima than e.g. the **Gauss–Newton** algorithm and the method of **gradient descent**.

Advanced Calibration – Nonlinear Distortions

- Radial distortions (Barrel)



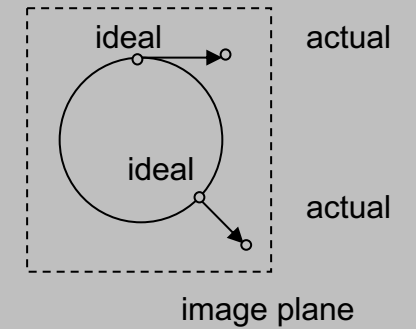
wideangle (barrel distortion)

Tangential distortions



tele (pincursion distortion)

Radial and Tangential Distortion



- Dealing with distortion parameters

- Usually the points of a captured checkerboard are radially distorted.

→ Alternatingly estimate

- Intrinsic parameters
- Distortion parameters

- **Notations** for projected corner points in normalized image coordinates and pixel coordinates:

Pixel image coordinates		Normalised Image coordinates	
$(\hat{u}, \hat{v})$	(u, v)	$(\hat{x}, \hat{y})$	(x, y)
distorted	non-distorted	distorted	non-distorted

- Usually **Brown's distortion model** is used to model radial distortions.

$$\begin{aligned} \hat{x} &= x + x [k_1(x^2 + y^2) + k_2(x^2 + y^2)^2] \\ \hat{y} &= y + y [k_1(x^2 + y^2) + k_2(x^2 + y^2)^2] \end{aligned} \quad \Rightarrow \quad \begin{aligned} \hat{x} &= x + x [k_1 r^2 + k_2 r^4] \\ \hat{y} &= y + y [k_1 r^2 + k_2 r^4] \end{aligned}$$

Advanced Calibration - Nonlinear Distortions

- Dealing with distortion parameters

Pixel image coordinates		Normalised Image coordinates	
$(\hat{u}, \hat{v})$	(u, v)	$(\hat{x}, \hat{y})$	(x, y)
distorted	non-distorted	distorted	non-distorted

- Assuming zero skew in the pinhole camera model leads to the following relations between points in normalized and pixel coordinates:

$$\begin{aligned} \hat{u} &= u_0 + \alpha \hat{x} \\ \hat{v} &= v_0 + \beta \hat{y} \end{aligned} \quad \begin{aligned} u &= u_0 + \alpha x \\ v &= v_0 + \beta y \end{aligned}$$

- Inserting the above relations into Brown's distortion model we get the following pair of equations for each point in each image:

$$\begin{aligned} \hat{u} - u &= (u - u_0)k_1r^2 + (u - u_0)k_2r^4 \\ \hat{v} - v &= (v - v_0)k_1r^2 + (v - v_0)k_2r^4 \end{aligned}$$

- Calculating the above equations for each point in each image and stacking them, we get the following system of equations:

$$\begin{pmatrix} \hat{u} - u \\ \hat{v} - v \end{pmatrix} = \begin{bmatrix} (u - u_0)r^2 & (u - u_0)r^4 \\ (v - v_0)r^2 & (v - v_0)r^4 \end{bmatrix} \begin{pmatrix} k_1 \\ k_2 \end{pmatrix} \Rightarrow \mathbf{d} = \mathbf{D}\mathbf{k}$$

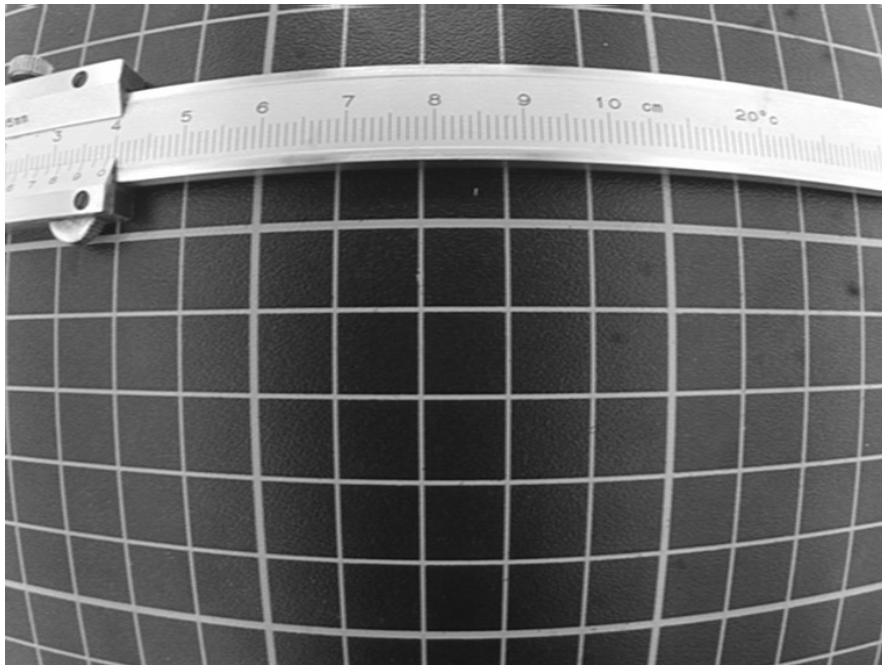
Distorted points
(detected)

Undistorted points
(projected from
camera model)

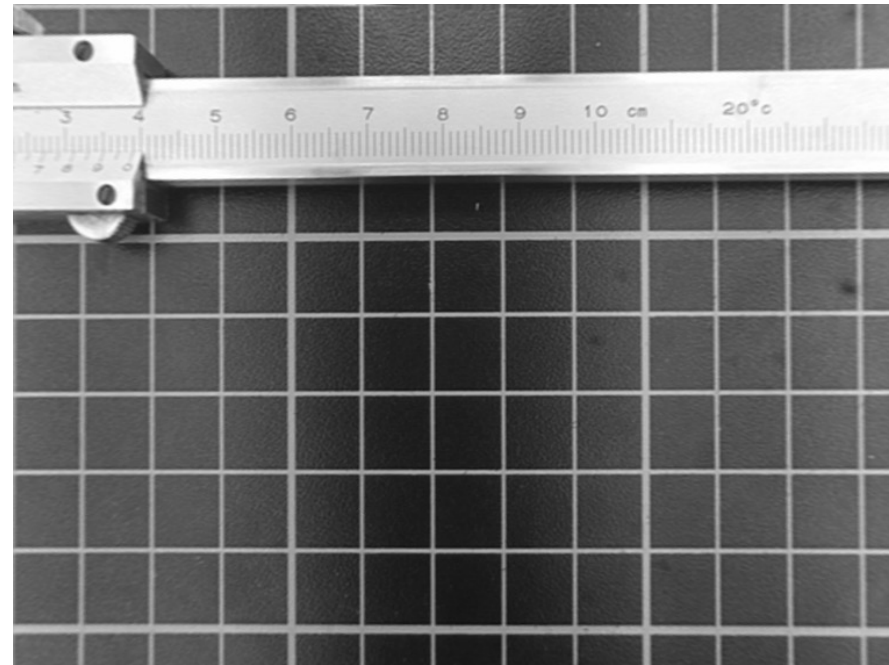
$$\Rightarrow \mathbf{k} = (\mathbf{D}^T \mathbf{D})^{-1} \mathbf{D}^T \mathbf{d}$$

Pseudo-inverse (see lecture
on „Parameter Estimation“)

Advanced Calibration – Image Rectification

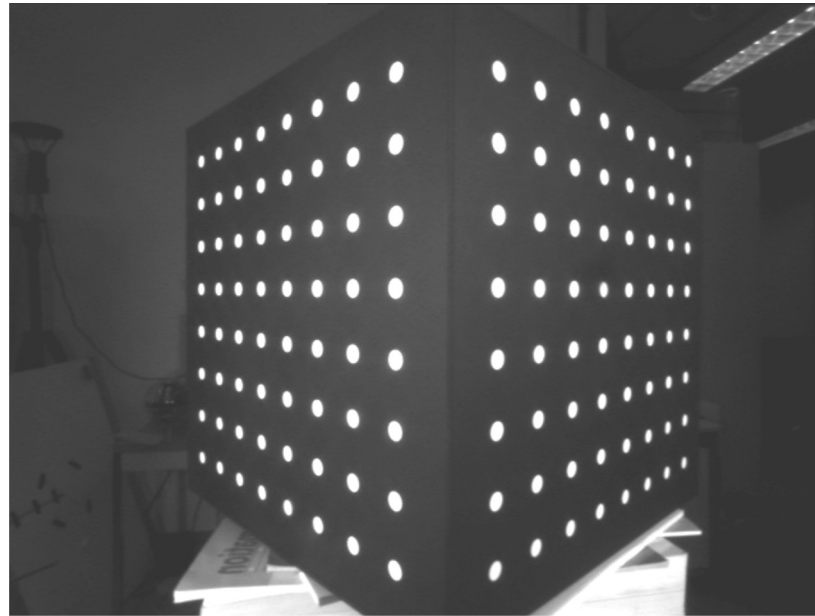
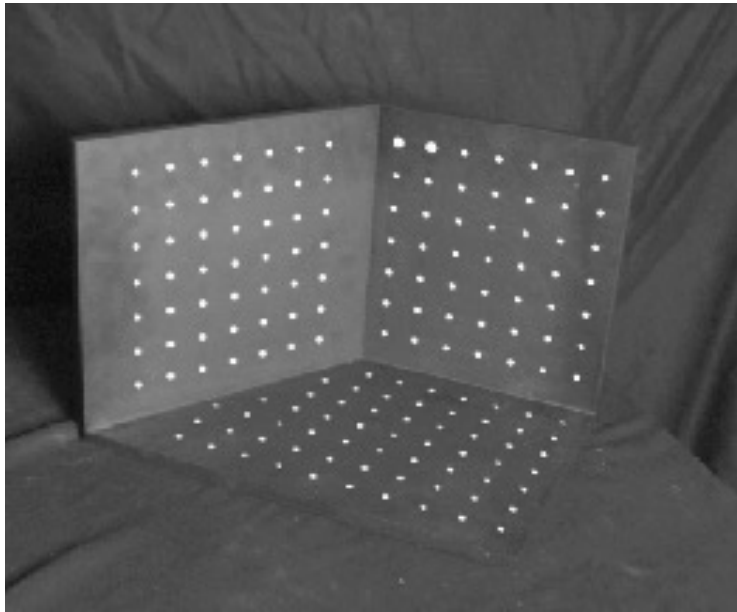


Distorted image.



Rectified image.

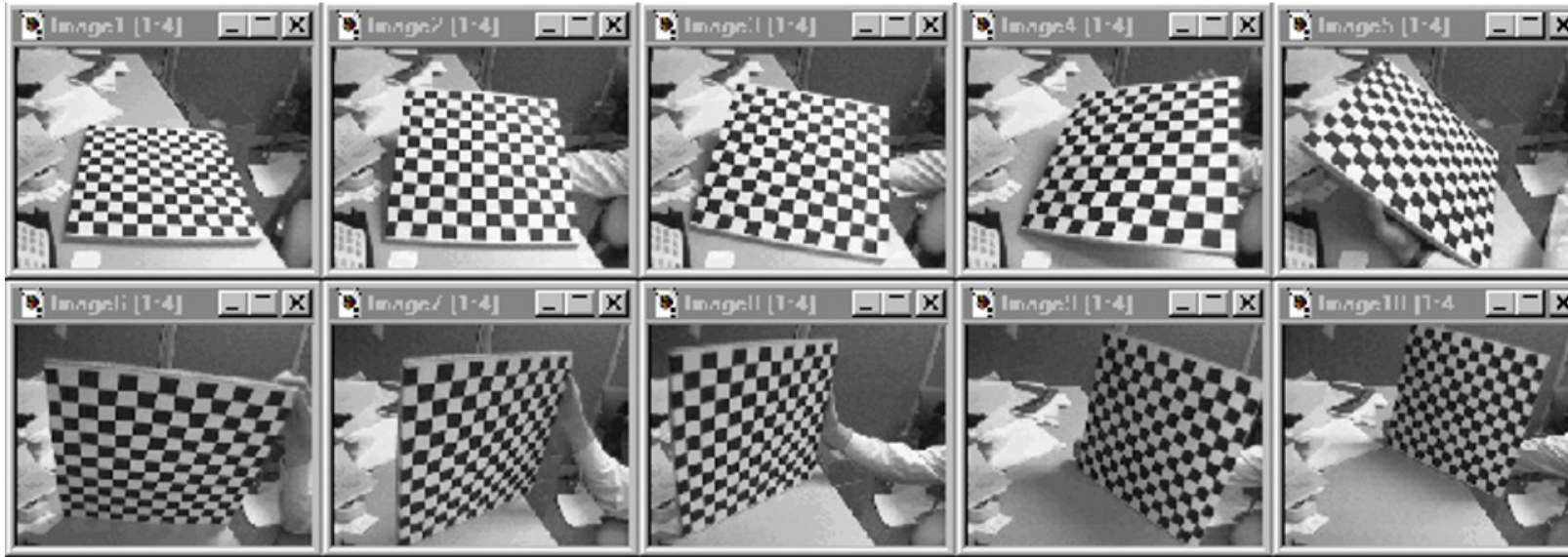
Calibration – Exemplarily Calibration Steps



- **Advantage**
 - Very simple to formulate and solve.
- **Disadvantages**
 - Doesn't tell you the camera parameters.
 - Doesn't model radial distortion.
 - Hard to impose constraints (e.g., known focal length).
 - Doesn't minimize the right error function.

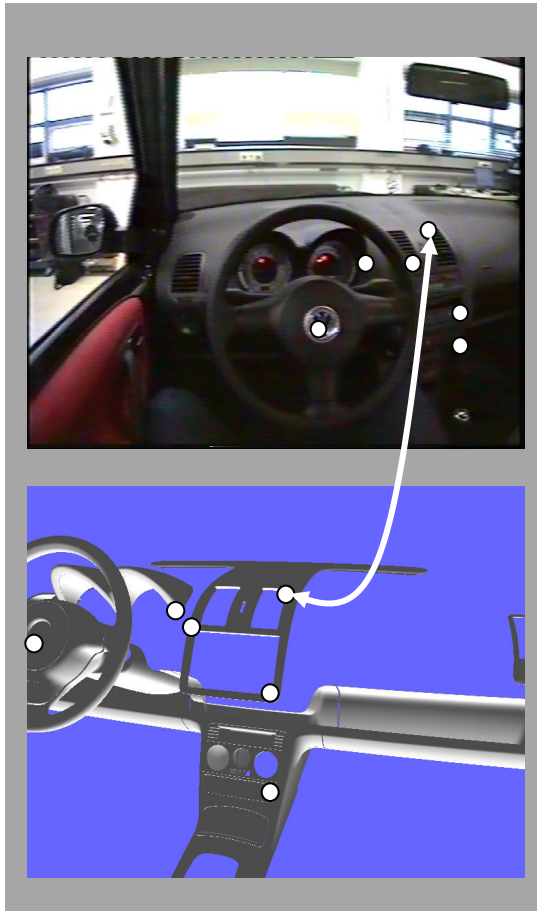
For those reasons nonlinear methods are preferred.
Nevertheless, the direct linear approaches can provide very good initializations.

Calibration – Other Approach

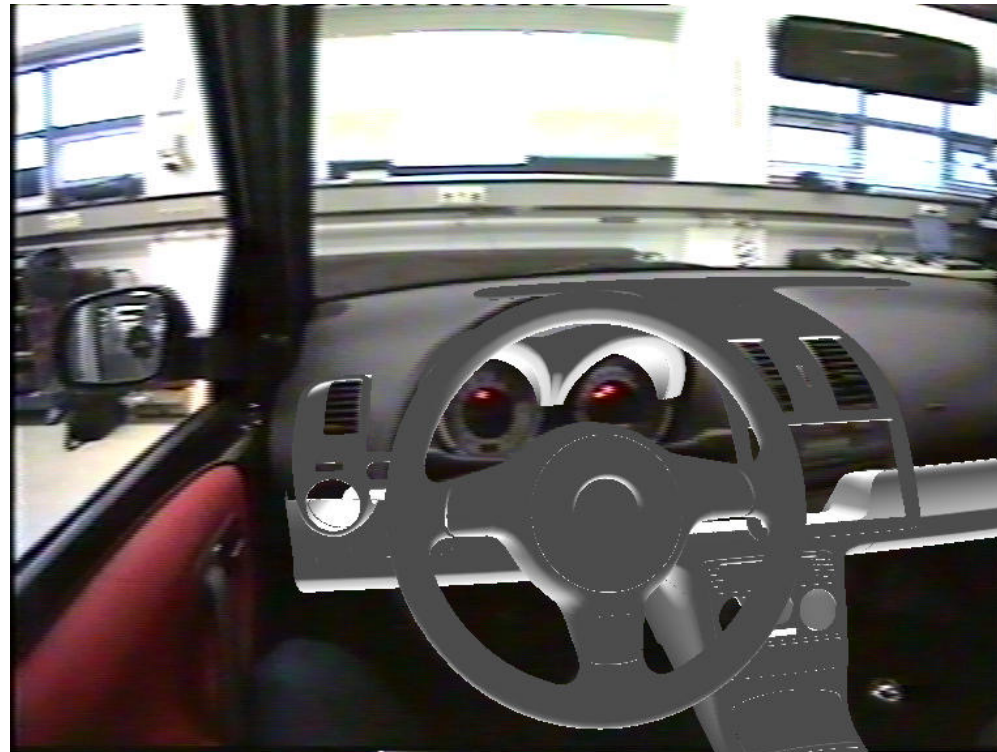


- **Calibration with a single plane:**
 - Only requires a plane.
 - Don't have to know positions/orientations.
 - Good code available online!
 - Intel's OpenCV library: <https://opencv.org/>
 - Matlab version by Jean-Yves Bouget: http://www.vision.caltech.edu/bougetj/calib_doc/index.html
 - Zhengyou Zhang's web site: <http://research.microsoft.com/~zhang/Calib/>

Application – Overlay of a virtual CAD-Cockpit onto a “real” image (Augmented Reality)



Collect 2D and 3D corresponding points (manual selection).



- Compute the P -matrix of the camera.
- Draw the picture as background.
- Use P to render the CAD object.

Important addendum: Ways to Represent Rotation

- **There are several options:**
 - 3D-Rotation matrix (3x3)
 - Euler angles
 - Axis angle (rotation vector)
 - Quaternions
 - ...

Rotation – 3D Rotation Matrix

- A matrix R is a rotation matrix if and only if it is an **orthogonal** matrix with **determinant +1**.
- Orthogonal matrix is a square matrix with real entries, where columns and rows are orthogonal vectors with length 1.

This is equivalent to:

$$RR^T = I \quad \text{or} \quad R^{-1} = R^T$$

Rotation – Euler Angle

- Euler's Theorem: Any two independent orthonormal coordinate frames can be related by a sequence of rotations (not more than three) about coordinate axes, where no two successive rotations may be about the same axis.
- A sequence of rotations around the principle axes is called an Euler Angle Sequence.
- We just multiply a sequence of rotation matrices to build a matrix from a set of Euler angles.

Rotation – Euler Angle

- **Euler angles for rotation $R = R_z R_y R_x$:**

- Rotation by ψ around z axis:

$$R_z = \begin{bmatrix} \cos \Psi & -\sin \Psi & 0 \\ \sin \Psi & \cos \Psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Rotation by θ around y axis:

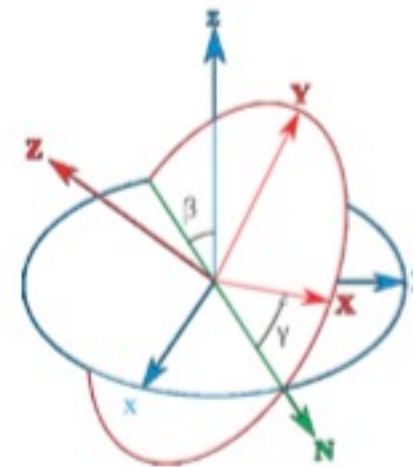
$$R_y = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

- Rotation by ϕ around x axis:

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

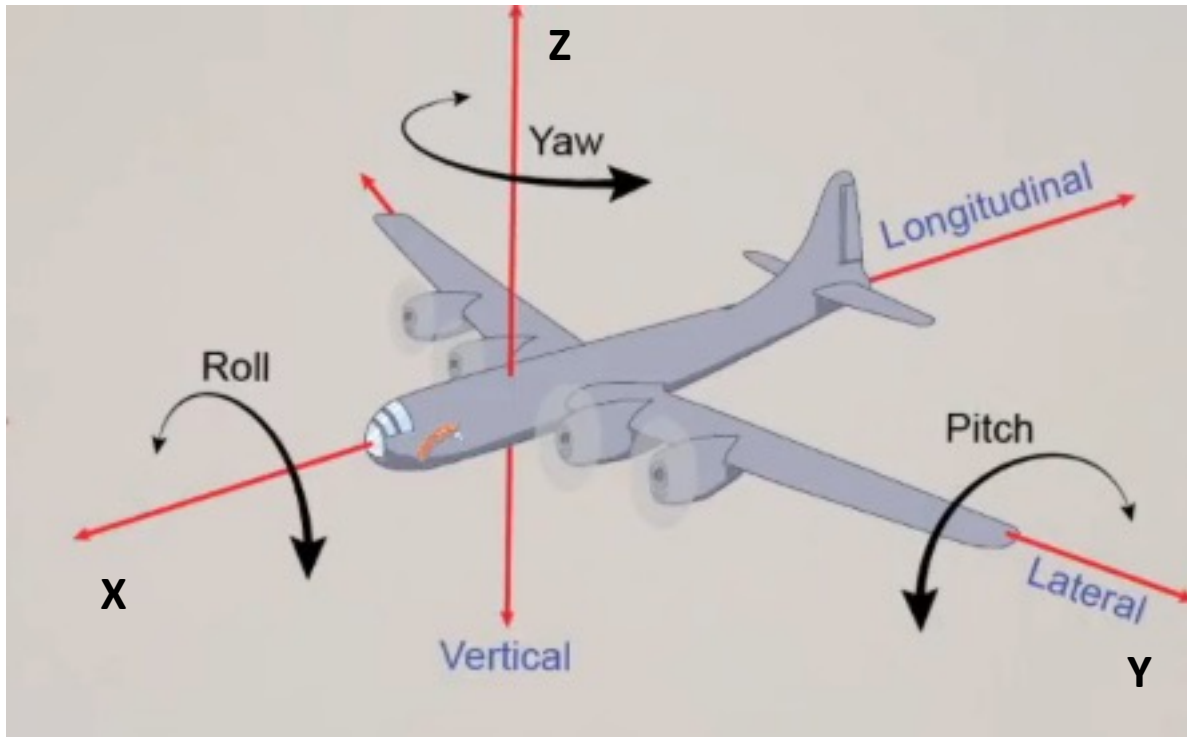
Rotation – Euler Angle

- **Properties of Euler angle representation:**
 - As matrix multiplication is not commutative, the order of operations is important.
 - Minimal parametrization, nice geometric interpretation.
 - Periodicity, Gimbal lock.



Rotation – Euler Angle

- Rotation angles like roll (around x-axis), pitch (around y-axis) and yaw (around z-axis) are often used in literature and in a lot of applications (vehicles, aircrafts, robotics, etc).



Rotation – Axis Angle

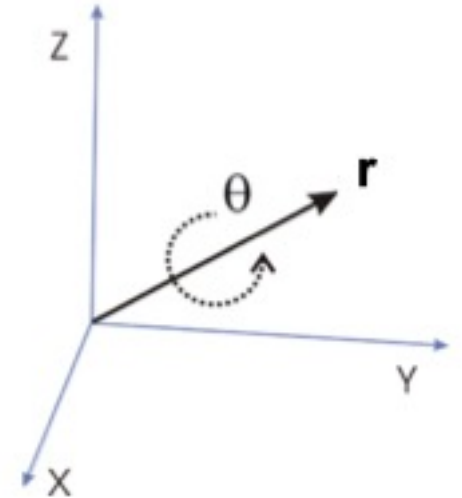
- **Represents rotation by a 3D vector:**

- Direction is rotation axis $\mathbf{r} = [a, b, c]^T$
- Length is rotation angle $\theta = \|\mathbf{r}\|$
- Conversion to rotation matrix by Rodrigues' formula:

$$R = \text{rot}(\mathbf{r}) = \mathbf{I}_3 + k(\theta) S(\mathbf{r}) + g(\theta) S(\mathbf{r})^2$$

$$\begin{aligned} k(\theta) &= \frac{\sin(\theta)}{\theta} \\ g(\theta) &= \frac{1 - \cos(\theta)}{\theta} \end{aligned} \quad S(\mathbf{v}) = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix}$$

- Singularity at $\theta = 0$.



- **Represents rotation by a 4D vector:**

- Quaternion is a 4D vector:

$$Q = q_w + q_x i + q_y j + q_z k \quad \Leftrightarrow \quad Q = q_w + \mathbf{q}_v$$
$$\mathbf{q}_v = q_x i + q_y j + q_z k = (q_x, q_y, q_z)$$

- Compromise between rotation matrix and minimal representation.
- Quaternions are actually an extension to complex numbers of the 4 components, one is a 'real' scalar number, and the other 3 form a vector in imaginary $i - j - k$ space.
- Singularity-free, can be interpolated.

- Conversion to rotation vector (axis-angle):

$$\mathbf{q} = (q_w, q_x, q_y, q_z)^T \rightarrow \mathbf{r} = \frac{1}{\sqrt{1 - q_w^2}} (q_x, q_y, q_z)^T$$
$$\theta = 2 \arccos(q_w)$$

- Conversion to rotation matrix:

$$\mathbf{R} = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}$$

THANK YOU!